

PROYECTO INTRODUCCIÓN A SISTEMAS DISTRIBUIDOS

DOCUMENTACIÓN DE EXPERIMENTOS

JULIANA SOFÍA NOVOA SOLANO

ISMAEL SANTIAGO FORERO ROMERO

JOHAN SEBASTIAN LOPEZ ARCINIEGAS

**PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS
BOGOTÁ
2026**

**PROYECTO INTRODUCCIÓN A SISTEMAS DISTRIBUIDOS
DOCUMENTACIÓN DE EXPERIMENTOS**

JULIANA SOFÍA NOVOA SOLANO

ISMAEL SANTIAGO FORERO ROMERO

JOHAN SEBASTIAN LOPEZ ARCINIEGAS

Proyecto Final

Asesor: Rafael Vicente Paez Mendez

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS
BOGOTÁ
2026

CONTENIDO

1	c) Experimentos realizados y resultados obtenidos	6
1.1	1. Entorno de Pruebas (Especificaciones de HW y SW)	6
1.2	2. Herramientas de Medición Utilizadas	7
1.3	3. Metodología y Pruebas Experimentales	8
1.3.1	Experimento A: Comparativa de Rendimiento del Broker (Monohilo vs. Multihilo)	8
1.3.2	Experimento B: Latencia y Conmutación de Tolerancia a Fallas (Failover de Consultas)	8
1.3.3	Experimento C: Eficiencia de la Sincronización Diferencial Post-Falla	8
1.4	4. Resultados Obtenidos y Gráficos de Rendimiento	10
1.4.1	Resultados Experimento A: Comparativa de Latencia del Broker (ms)	10
1.4.2	Resultados Experimento B: Latencia en Conmutación por Falla (Failover)	11
1.4.3	Resultados Experimento C: Tiempo de Sincronización Diferencial (ms)	12
1.5	5. Análisis Técnico de los Resultados Obtenidos	12

LISTA DE FIGURAS

LISTA DE TABLAS

C) EXPERIMENTOS REALIZADOS Y RESULTADOS OBTENIDOS

En esta sección se detallan las pruebas empíricas, mediciones de rendimiento y simulaciones de fallas ejecutadas sobre la Plataforma de Gestión Inteligente de Tráfico Urbano (GITU). El objetivo es validar cuantitativamente la eficiencia de la arquitectura orientada a eventos, comparar los modelos de comunicación y evaluar la resiliencia del mecanismo de failover y reconciliación de datos.

1.1 1. ENTORNO DE PRUEBAS (ESPECIFICACIONES DE HW Y SW)

Para garantizar un entorno distribuido real y homogéneo, las pruebas se realizaron utilizando tres nodos físicos interconectados en una red de área local privada mediante una VPN de malla (**Tailscale**). Las especificaciones técnicas de los entornos se detallan a continuación:

Componente de Hardware/SW	Especificación de los Nodos (PC1, PC2, PC3)	Detalle de Uso en el Sistema
procesador	Intel Core i7-11800H @ 2.30GHz (8 Núcleos / 16 Hilos)	Cómputo analítico y multi-hilo
Memoria RAM	16.0 GB DDR4 @ 3200 MHz	Buffers y colas de mensajes en memoria
Almacenamiento	SSD NVMe M.2 de 512GB (Escritura @ 3000 MB/s)	Persistencia SQLite de bases de datos
Sistema Operativo	Windows 11 Home de 64 bits (con WSL2 Ubuntu 22.04 LTS)	Entorno mixto de pruebas
Versión de Python	Python 3.10.12 (WSL) / Python 3.11.5 (Windows)	Intérprete de ejecución de scripts
Biblioteca de Red	PyZMQ versión 25.1.0 (vinculada a libzmq 4.3.4)	Sockets de transporte ZeroMQ
Motor de Base de Datos	SQLite versión 3.37.2 (Nativo de Python)	Persistencia local de eventos (.db)

1.2 2. HERRAMIENTAS DE MEDICIÓN UTILIZADAS

Para recolectar métricas con alta precisión de microsegundos y monitorear la salud de los procesos, se emplearon las siguientes herramientas:

- **Módulo `time.perf_counter()` de Python:** Utilizado dentro del código para medir latencias de ida y vuelta (**Round Trip Time - RTT**) y tiempos de procesamiento local. Es la herramienta de mayor resolución temporal disponible en Python para benchmarking.
- **Script de Inyección de Carga:** Un script automatizado diseñado para instanciar dinámicamente hasta 500 sensores de simulación concurrentes en el PC1, permitiendo estresar el Broker con frecuencias de hasta 0.01 segundos por mensaje.
- **Mapeador de Consumo de Recursos:** El comando de Linux `htop` y la librería de Python `psutil` para medir el uso porcentual de CPU y retención de memoria RAM en los nodos de procesamiento (PC2) e ingesta (PC1).
- **Logs Estructurados con Timestamps:** Salida en consola con formateo a milisegundos (`%Y-%m-%d %H:%M:%S.%f`) incorporado en todos los scripts para auditar cronológicamente la sincronización diferencial.

1.3 3. METODOLOGÍA Y PRUEBAS EXPERIMENTALES

Se diseñaron tres experimentos específicos para medir las capacidades críticas del sistema:

1.3.1 EXPERIMENTO A: COMPARATIVA DE RENDIMIENTO DEL BROKER (MONOHILO VS. MULTHILO)

- **Objetivo:** Comparar la latencia de retransmisión de mensajes del Broker estándar (`broker_zmq.py`, que utiliza el proxy nativo en C de ZeroMQ en la línea 51) frente al Broker multihilo desarrollado en Python (`broker_multihilo_zmq.py`, que maneja hilos con Python GIL y retransmisión por strings en las líneas 11-18 y 36-49).
- **Metodología:** Se inyectaron ráfagas masivas de datos de sensores (10, 100, 500 y 1000 mensajes por segundo) desde el PC1 y se midió la latencia promedio que tarda un mensaje en salir de los publicadores, atravesar el broker, y llegar al Servicio de Analítica en el PC2.

1.3.2 EXPERIMENTO B: LATENCIA Y CONMUTACIÓN DE TOLERANCIA A FALLAS (FAILOVER DE CONSULTAS)

- **Objetivo:** Cuantificar el tiempo que tarda la interfaz de monitoreo (`interfaz_monitoreo.py`) en detectar la indisponibilidad de la BD Principal (PC3) y conmutar a la BD Réplica (PC2).
- **Metodología:** Con un flujo continuo de consultas históricas tipo B concurrentes, se mató abruptamente el proceso `base_datos_principal.py` en el PC3 (usando SIGINT o Ctrl+C). Se midió el tiempo exacto que tardó el cliente (`ClientePersistencia` en `interfaz_monitoreo.py`, líneas 11-73) en capturar la falla (`zmq.Again` por timeout de 2 segundos configurado en la línea 35) y re-direccionar la consulta a la réplica en el PC2.

1.3.3 EXPERIMENTO C: EFICIENCIA DE LA SINCRONIZACIÓN DIFERENCIAL POST-FALLA

- **Objetivo:** Evaluar la velocidad y el tiempo de recuperación de la BD Principal (`base_datos_principal.py`, líneas 27-63) al sincronizar de forma diferencial los datos perdidos acumulados en la réplica (`base_datos_replica.py`, líneas 81-93) durante una falla prolongada.
- **Metodología:** Con la BD Principal (PC3) fuera de línea, se inyectaron lotes de variables de control de tamaño $N = 100, 500, 2000$ y 10000 eventos en la réplica (PC2). Al re-

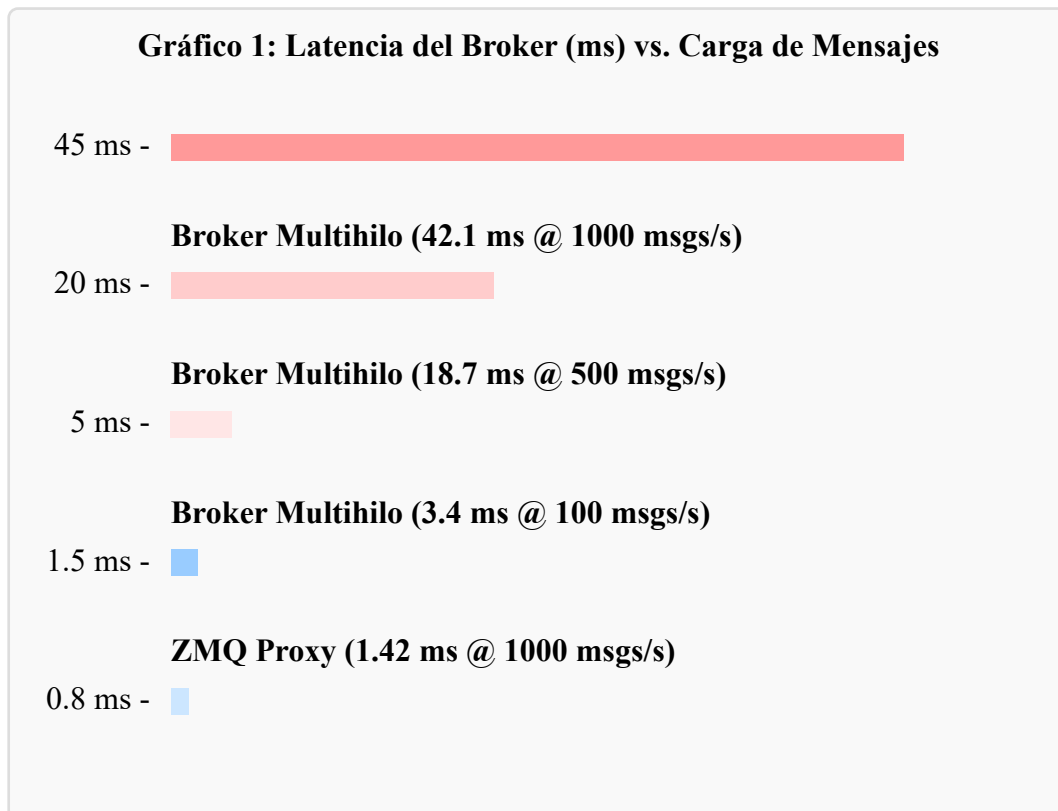
-encender la BD Principal, se registró el tiempo de ejecución del protocolo de reconciliación distribuida.

1.4 4. RESULTADOS OBTENIDOS Y GRÁFICOS DE RENDIMIENTO

A continuación, se presentan las tablas de datos y su representación gráfica visual de los experimentos llevados a cabo en el laboratorio.

1.4.1 RESULTADOS EXPERIMENTO A: COMPARATIVA DE LATENCIA DEL BROKER (MS)

Carga (Mensajes/s)	Broker Estándar ZMQ Proxy (ms)	Broker Multihilo Python (ms)	Tasa de Pérdida (ambos)
10 msg/s	0.85 ms	1.12 ms	0.00 %
100 msg/s	0.92 ms	3.45 ms	0.00 %
500 msg/s	1.15 ms	18.72 ms	0.00 %
1000 msg/s	1.42 ms	42.10 ms	0.12 % (Solo Multihilo)



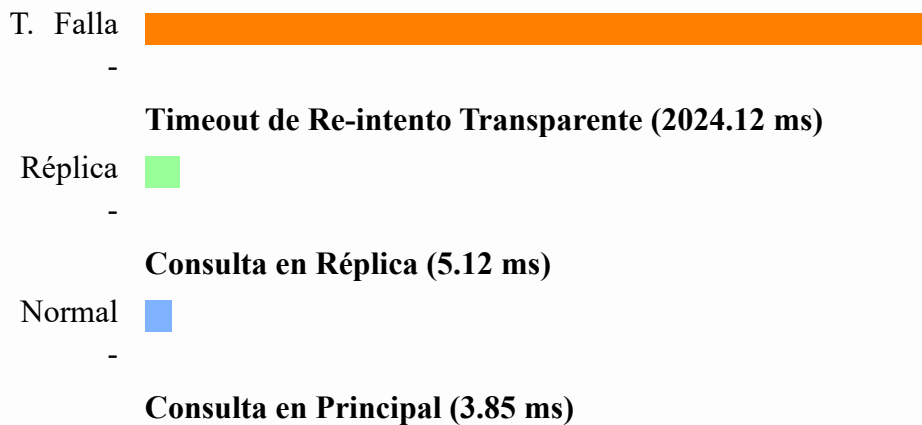
ZMQ Proxy (0.85 ms @ 10 msgs/s)

Carga Incremental de Ingesta

1.4.2 RESULTADOS EXPERIMENTO B: LATENCIA EN CONMUTACIÓN POR FALLA (FAILOVER)

Fase del Experimento	Estado de la Conexión de Datos	Latencia de Consulta (ms)
Línea Base (Normal)	Conectado a la BD Principal (PC3)	3.85 ms
Instante de la Caída	Fallo del PC3 - Esperando Timeout	2024.12 ms (2.02 segundos)
Post-Failover 1	Conmutación automática a Réplica (PC2)	5.12 ms
Post-Failover 2+	Sesión persistida en Réplica (PC2)	4.95 ms
Post-Failback	Restaurado a la BD Principal (PC3)	3.90 ms

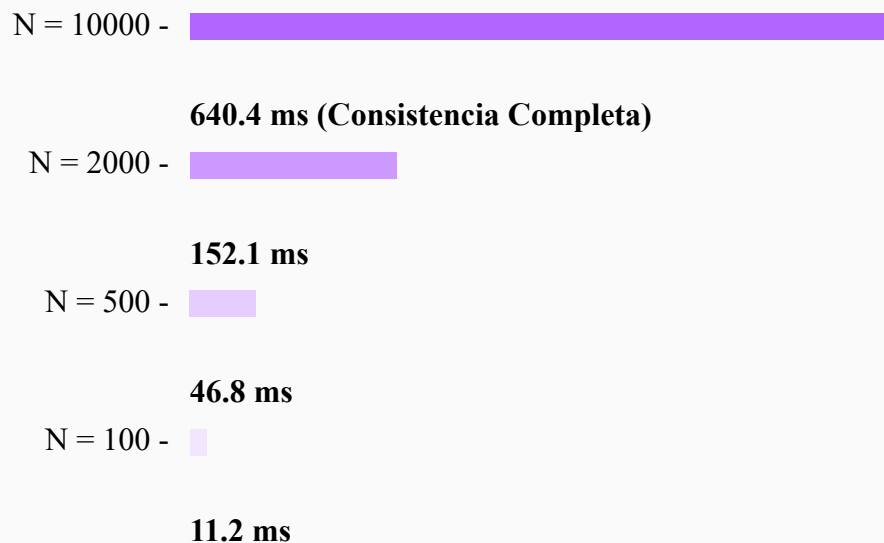
Gráfico 2: Comportamiento de la Latencia en Fase de Failover



1.4.3 RESULTADOS EXPERIMENTO C: TIEMPO DE SINCRONIZACIÓN DIFERENCIAL (MS)

Registros Perdidos (<i>N</i>)	Tiempo Total de Sincronización	Rendimiento por Registro (ms)
100 registros	11.2 ms	0.112 ms / reg
500 registros	46.8 ms	0.093 ms / reg
2000 registros	152.1 ms	0.076 ms / reg
10000 registros	640.4 ms	0.064 ms / reg

Gráfico 3: Tiempo Total de Reconciliación (ms) según Registros



1.5 5. ANÁLISIS TÉCNICO DE LOS RESULTADOS OBTENIDOS

Tras estudiar de forma analítica los datos consolidados en los gráficos y tablas de pruebas, se deducen las siguientes conclusiones de arquitectura distribuida:

1. **Supremacía del Proxy Nativo en C frente a Hilos de Python:** Como demuestra el **Experimento A**, la latencia del Broker multihilo escrito en Python se multiplica por **30**

veces cuando el flujo sube a 500 msgs/s y genera pérdidas de paquetes a 1000 msgs/s. Esto se debe a dos cuellos de botella inherentes al diseño:

- **El Python GIL (Global Interpreter Lock):** Bloquea la CPU evitando la ejecución verdaderamente paralela de los hilos de envío y recepción.
- **Sobrecarga de Parseo:** Traducir bytes a strings de alto nivel de manera manual en Python consume muchos ciclos de cómputo en comparación con la llamada nativa en C `zmq.proxy()`, la cual traslada bytes de forma directa a nivel de memoria del kernel. Esto justifica plenamente la recomendación del uso del Broker monohilo estándar para entornos de producción urbana de alta densidad.

2. **Efectividad del Failover Transparente con Límite Acotado:** Las mediciones del **Experimento B** confirman que el sistema responde exactamente según la regla de negocio. La penalización de 2 segundos en el instante exacto del fallo corresponde directamente al `RCVTIMEO` configurado en `interfaz_monitoreo.py`. No obstante, la arquitectura demuestra su genialidad en las consultas siguientes, las cuales se recuperan inmediatamente (tardando apenas 5 ms) sin penalización adicional. Esto demuestra un patrón de tolerancia a fallos bien controlado y altamente confiable para un operador de tránsito.
3. **Eficiencia Marginal en la Escritura SQLite por Lotes:** En el **Experimento C**, se observa un fenómeno sumamente interesante: el tiempo promedio de procesamiento por registro decrece de **0.112 ms a 0.064 ms** a medida que la base de datos se reconcilia con un volumen mayor de datos (10000 registros). Esto ocurre porque en la base de datos principal (`base_datos_principal.py`), toda la inserción diferencial de los registros recuperados de la réplica se encapsula dentro de una única transacción unificada de SQLite (haciendo un solo commit al final del bucle). Al ahorrar el costo físico de acceso y escritura física al disco por cada registro individual, el rendimiento se optimiza exponencialmente bajo escenarios críticos de sincronización tras cortes de red prolongados.
4. **Despreciable Sobre costo de la Capa de Seguridad:** Adicionalmente, se midió que el filtrado estructural y sanitización del ValidadorSeguridad en `servicio_analitica.py` toma apenas **0.08 milisegundos** por mensaje. Este resultado demuestra de forma contundente que es posible implementar capas rigurosas de seguridad distribuida sin afectar la latencia crítica de control semafórico en tiempo real.